

Lecture 09: Semaphores

Vivek Kumar

Computer Science and Engineering

IIIT Delhi

vivekk@iiitd.ac.in

Last Lecture

```

int main() {
    int fd[2], status;
    pipe(fd);
    if(fork() == 0) {
        /* Child process */
        close(fd[0]);
        char buff[] = "Hello my dear good Parent";

        write(fd[1], buff, sizeof(buff));
        exit(0);
    }
    /* Parent process */
    close(fd[1]);
    char buff[100];
    read(fd[0], buff, sizeof(buff));
    printf("My obedient child says: %s\n", buff);
    wait(NULL);
    return 0;
}

```

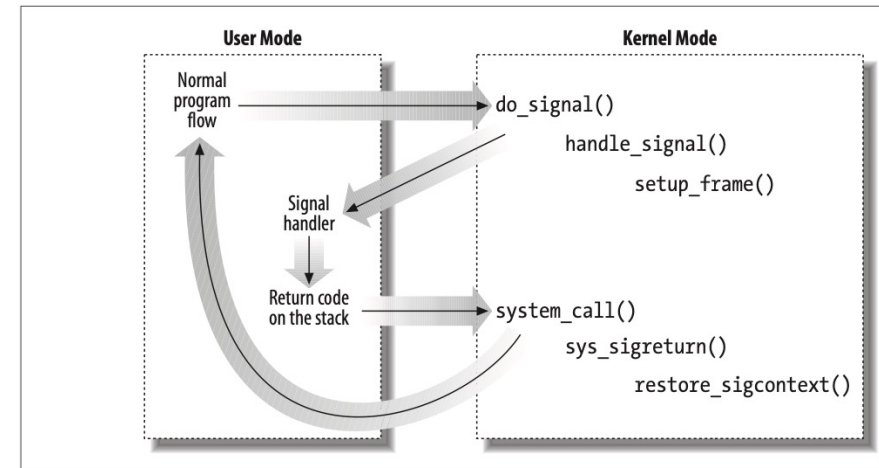
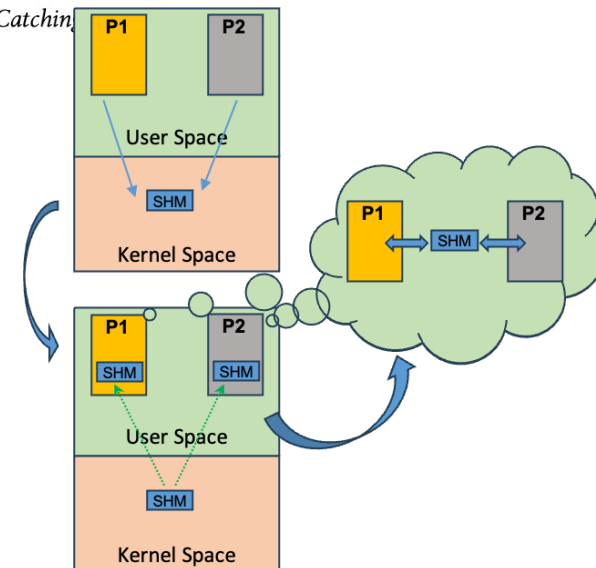


Figure 11-2. Catching



Today's Class

- Semaphores for process synchronization
 - Mutual exclusion
 - Critical section
- Producer consumer problem

The OS Story Evolving So Far...

1. A **cook** (OS) manages a **kitchen** (*computer*)
2. The job of a cook is to do **cooking** (*execute binaries*) by reading **recipes** (*executable file*) in a **standard format** (*ELF*)
3. The cook has received recipe of a dish in **different files** (*relocatable files*). It will have to use a **stapler** (*linker*) to stitch these files to make a final recipe
4. The cook has to **store** (*load*) the **steps** (*instructions*) from the recipe in his memory to start cooking. The cook then execute instructions **one by one** (*call stack*) for cooking
5. The kitchen has gained a lot of popularity and have started receiving lot of recipes for cooking. The cook has hired several **helpers** (*processes*), where one helper cooks one recipe
6. To ensure the helpers don't fight for kitchen resources and make a mess, the ground rule is set that all helpers must talk to the head cook via a **hotline** (*interrupts*) if they need to access any kitchen resources
7. One of the helper got a huge recipe to cook. He decided to create an **identical twin** (*fork*) to let him own the **preparation** (*exec*) of one of the sub recipe

The OS Story Evolving So Far...

8. The identical twin of a helper and the helper himself must be able to **communicate** (*inter-process communication*) with each other without disturbing other helpers for properly cooking the recipe
9. There is a **race condition** between the identical twin of a helper and the helper himself for using a single frying pan, as it has to be **shared** between themselves. They use a special coin as a **token** (**semaphore**) which one of them must hold to gain access to the frying pan

Array Sum Program (Version 1)

```
int main() {
    shm_t* shm = setup();
    if(fork()==0) {
        int local=0;
        for(int i=0; i<SIZE/2; i++) shm->sum1+=shm->array[i];
        cleanup_and_exit();
    } else {
        int local=0;
        for(int i=SIZE/2; i<SIZE; i++) shm->sum2+=shm->array[i];
        wait(NULL);
    }
    int total = shm->sum1 + shm->sum2;
    cleanup();
    return 0;
}
```

```
typedef struct shm_t {
    int A[SIZE];
    int sum1;
    int sum2;
} shm_t;
```

- Both child and the parent process calculate the partial sum independently
- Parent process combine the partial sum from the child to its own calculation to get the total
 - Done only after **wait**

Array Sum Program (**Version 2**)

```

int main() {
    shm_t* shm = setup();

    int chunks = SIZE/NPROCS;
    for(int i=0; i<NPROCS; i++) {
        if(fork()==0) {
            int local=0;
            int start = i*chunks;
            int end = start+chunk;
            for(int i=start; i<end; i++) local += shm->array[i];

            shm->sum += local;

            cleanup_and_exit();
        }
    }
    for(int i=0; i<NPROCS; i++) wait(NULL);
    cleanup();
    return 0;
}

```

```

typedef struct shm_t {
    int A[SIZE];
    int sum;
} shm_t;

```

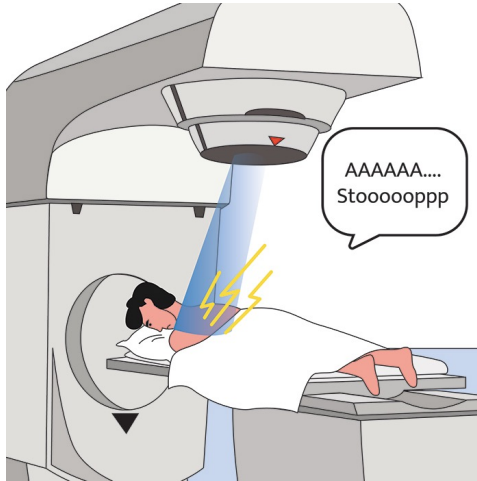
- We want more than one process to participate in the parallel array sum computation
- Children update the global **sum** in the shared memory region
 - **Is is correct?**
 - **Race condition!**
 - All children are updating the same shared global variable sum

Race Condition

```
shm->sum += local;
```

- Final value of the sum will not be correct as there is a race between reading and writing the counter “sum” by multiple processes
- The line of code shown above is called as **critical section**
 - **Critical section** is a line **or** block of code that access shared modifiable data or resource that should be operated on by only one process at a time

Real World Accidents From Race Conditions

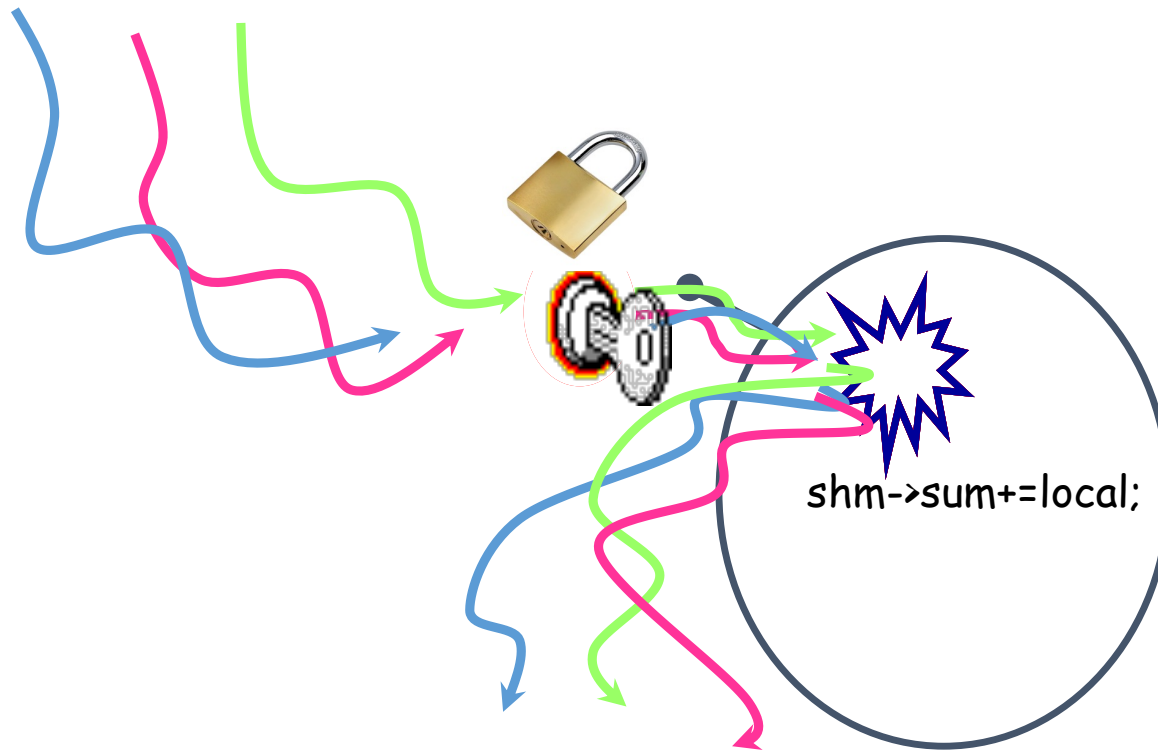


- Therac-25 radiation overdose in 1980s
 - Radiation overdose as the software failed to detect when the operator finished editing due to race condition. It resulted in several deaths and severe injuries
- Northeastern blackout of 2003
 - Race conditions failed to notify the operators about local power line failure, causing the problem to spread across the power grid
- NASDAQ software glitch in 2012
 - A race condition prevented the delivery of order Facebook IPO confirmations, so those orders were re-submitted repeatedly

Mutual Exclusion

- **Mutual exclusion** is a property that ensures that there is no race condition by executing the critical section by a single process only at any given time
 - One way to achieve it is by using locks 
 - We will revisit this topic during lectures on concurrency

Visualizing Mutual Exclusion



- Each process must acquire a lock before entering a critical section
- A “Lock” should allow only one process to enter the critical section
- Rest all processes should queue (wait) to get the key
- The process acquiring the lock must release it when exiting the critical section

Naïve Implementation of a Lock (1/3)

```
int value = FREE //inside SHM
```



```
Acquire() {
    Disable interrupts
    while (value == BUSY) {
        sleep
        // Process moved to wait queue
    }
    value = BUSY
    Enable interrupts
}
```

```
Release() {
    Disable interrupts
    if (anyone in wait queue) {
        Move a process into ready queue
    }
    value = FREE
    Enable interrupts
}
```

- Goal: achieving mutual exclusion over a critical section when multiple processes are going to execute that critical section
 - **Let us assume a uniprocessor system for simplicity**
- A naïve way is to let a process complete the execution of a critical section without interrupting it
- **Any issues here?**
 - The sleeping process has disabled process scheduling!

Naïve Implementation of a Lock (2/3)

```
int value = FREE //inside SHM
```



```
Acquire() {  
    Disable interrupts  
    while (value == BUSY) {  
        Enable interrupts and sleep  
        // Process moved to wait queue  
        Disable interrupts  
    }  
    value = BUSY  
    Enable interrupts  
}
```

```
Release() {  
    Disable interrupts  
    if (anyone in wait queue) {  
        Move a process into ready queue  
    }  
    value = FREE  
    Enable interrupts  
}
```

● Solution?

- The sleeping process must enable interrupts before going to sleep and must disable it after coming out of sleep

Naïve Implementation of a Lock (3/3)

```
int value = FREE //inside SHM
```



```
Acquire() {  
    Disable interrupts  
    while (value == BUSY) {  
        Enable interrupts and sleep  
        // Process moved to wait queue  
        Disable interrupts  
    }  
    value = BUSY  
    Read a story book?  
    Enable interrupts  
}
```

```
Release() {  
    Disable interrupts  
    if (anyone in wait queue) {  
        Move a process into ready queue  
    }  
    value = FREE  
    Enable interrupts  
}
```

- Any issues?
 - Process acquiring the lock will enjoy an everlasting vacation on the CPU
- Critical section must be as small as possible

Atomic Read-Modify-Write Instructions

- Process updating a variable will have to first “Read” (R) the variable, followed by “Modify” (M) and finally “Store” (S) the updated value so as to be visible to other processes
 - Any issues?
 - $P1(\text{Var}) \rightarrow R; P2(\text{Var}) \rightarrow M; P3(\text{Var}) \rightarrow S$
 - Overlapping RMS!
- How to fix using hardware support?
 - Hardware can combine RMS as a single instruction for a specific type of variables (**Atomic**)
 - Uninterruptible
 - Atomic instructions read a value from memory and write a new value atomically
 - E.g., compare and swap (CAS), atomic increment/decrement, atomic load/store etc.
 - Hardware is responsible for implementing this correctly

Mutual Exclusion using CAS (1/2)

```
int value = FREE //inside SHM
```



```
Acquire() {  
    while (CAS(value, FREE, BUSY) != true);  
}
```

```
Release() {  
    value = FREE  
}
```

- CAS atomically checks if the **value** is **FREE** and set it to **BUSY** if currently **FREE**
 - Returns **true**, otherwise **value** remains unchanged and returns **false**
- **Any issues?**
 - Busy waiting!
 - Process wastes CPU cycles carrying out constant checks until the lock is free
 - **What should the process do when it finds the value!=FREE?**

Mutual Exclusion using CAS (2/2)

```
int value = FREE //inside SHM
```



```
Acquire() {  
    while (CAS(value, FREE, BUSY) != true) {  
        sleep();  
    }  
}
```

```
Release() {  
    value = FREE  
}
```

- Forcing the process to sleep will move it to the wait queue
- **Any issues?**
 - Sleep for how long?
- **Desirable scenario**
 - If any process fails to acquire the lock then it goes to sleep but is awakened by the process calling the release
 - Can be achieved using **Semaphores!**

Semaphores

Value of Semaphore	Process-T1
1	
1	call sem_wait()
0	sem_wait() returns
0	(crit sect)
0	call sem_post()
1	sem_post() returns

Val	Process-T1	State	Process-T2	State
1		Run		Ready
1	call sem_wait()	Run		Ready
0	sem_wait() returns	Run		Ready
0	(crit sect begin)	Run		Ready
0	Interrupt; Switch→T1	Ready		Run
0		Ready	call sem_wait()	Run
-1		Ready	decr sem	Run
-1		Ready	(sem<0)→sleep	Sleep
-1		Run	Switch→T0	Sleep
-1	(crit sect end)	Run		Sleep
-1	call sem_post()	Run		Sleep
0	incr sem	Run		Sleep
0	wake(T1)	Run		Ready
0	sem_post() returns	Run		Ready
0	Interrupt; Switch→T1	Ready		Run
0		Ready	sem_wait() returns	Run
0		Ready	(crit sect)	Run
0		Ready	call sem_post()	Run
1		Ready	sem_post() returns	Run

- A semaphore is an object with an integer **value** (user initialized) that could be manipulate with:

- **sem_wait**
 - value = value **-1** (atomically!)
 - Calling process blocks (sleep) if value < 0, otherwise non-blocking
- **sem_post**
 - Non-blocking API
 - value = value **+1** (atomically!)
 - If there are one more processes blocked inside sem_wait then wake **any one** of them (some OS might wake in FIFO order, but its not universal)
- Helps in achieving mutual exclusion over a critical section
 - Uses atomic instructions internally

Array Sum Program (Version 2)

```
int main() {
    shm_t* shm = setup();
    sem_init(&shm->mutex, 1, 1);
    int chunks = SIZE/NPROCS;
    for(int i=0; i<NPROCS; i++) {
        if(fork()==0) {
            int local=0;
            int start = i*chunks;
            int end = start+chunk;
            for(int i=start; i<end; i++) local += shm->array[i];
            sem_wait(&shm->mutex);
            shm->sum += local;
            sem_post(&shm->mutex);
            cleanup_and_exit();
        }
    }
    for(int i=0; i<NPROCS; i++) wait(NULL);
    cleanup();
    return 0;
}
```

```
typedef struct shm_t {
    int A[SIZE];
    int sum;
    sem_t mutex;
} shm_t;
```

- 1) munmap (shm, ...)
- 2) close (...)
- 3) shm_unlink (...)
- 4) sem_destroy

- We used a **Binary semaphore** to synchronize the accesses on the sum variable
 - Waiting processes are added into a FIFO queue and removed one by one after each sem_post
- Semaphore helped in achieving mutual exclusion over the critical section
 - No more race condition!
- There are counting semaphores as well
 - Initialized with value > 1
 - To handle access to shared resource where more than one process is allowed to operate simultaneously (e.g., resource pool)

Measuring the Parallel Performance?

- Speedup is used to denote the performance improvement by using multiple processes
 - $\text{Speedup} = T_{\text{serial}} / T_{\text{parallel}}$

Next Lecture

- Producer and consumer problem